COMP2040 – Operating Systems

C Concepts for Lab 03: Structs, Arrays and Pointers

The Lab 03 simulator `sched_sim.c` uses some C that Lab 02 did not: structs, an array of arrays, and pointers to structs. This guide explains each idea with a few lines taken from the starter file and says where to find them. Read it before the lab, and keep it open while you read the starter. (The *C Primer for Lab 02* covers the basics: arrays, pointers and `&`.)

1 Constants and variables

#define gives a number a name. Before compiling, the preprocessor replaces every `MAX_PROCS` with `10`. Change the number once and every array that uses it follows. A `#define` line has no `=` and no `;`.

```
#define MAX_PROCS  10
static process_t procs[MAX_PROCS];
```

Global and local variables. A variable declared *outside* every function is *global*: all functions in the file can use it, it lives for the whole run, and it starts at `0` automatically (`static` means "only in this file"). A variable declared *inside* a function is *local*: it exists only during that call, and you must give it a value yourself.

```
static int n_slices;          /* global: every function can use it, starts at 0 */

static void schedule_nonpreemptive(int use_sjf) {
    int time = 0;             /* local: only inside this call */
```

Special values. Times are never negative, so `start = -1` means "not started yet"; process numbers start at 1, so `last_id == 0` means "no process has run yet". **const** marks data that must not change, such as the workload tables: the compiler reports an error if code tries to write to it.

2 Arrays and arrays of arrays

A 1D array holds a fixed number of values of one type; indexes start at `0`. `procs` always has 10 slots, and only the first `n` are used.

An array of arrays (2D array). Each workload is an array of *rows*, and each row is itself an array of 2 ints, `{arrival, burst}`. The first index chooses the row (the process), the second the column.

```
static const int lab_workload[][2] = {
    {0, 5}, {1, 3}, {3, 6}, {5, 1}, {16, 2},
};
```

	column 0 (arrival)	column 1 (burst)	
row 0	0	5	P1
row 1	1	3	P2: `lab_workload[1][0]` is 1
...			
row 4	16	2	P5

`[2]` fixes the length of every row; the empty `[]` lets the compiler count the rows (5). In memory the rows are stored one after another: `0 5 1 3 3 6 5 1 16 2`.

Counting the rows with sizeof. `sizeof` gives a size in bytes: the whole table is 40 bytes and one row (`lab_workload[0]`) is 8, so the division gives 5 rows. Adding a row to the table needs no other change.

```
n = (int)(sizeof lab_workload / sizeof lab_workload[0]);
```

A pointer to a row. `load_workload()` must work with either table, so it keeps a pointer to whichever one was chosen and then uses it exactly like the 2D array. Read the declaration from the name outwards: `table` is a *pointer* to an *array of 2* `const int`. The parentheses matter: `const int *table[2]` would be an array of 2 pointers.

```
const int (*table)[2];
table = lab_workload;          /* point at the first row */
procs[i].burst = table[i][1];  /* row i, column 1 */
```

In sched_sim.c: `load_workload()`.

An array of pointers. Each slot of the round-robin queue holds the *address* of a process, not a copy. Working through that address changes the real process in `procs[]`.

```
static process_t *queue[MAX_PROCS];
queue_push(&procs[i]);         /* store the address of process i */
```

In sched_sim.c: the queue functions and `admit_arrivals()`.

3 Structs

A struct groups related variables (its *fields*) into one value. `typedef` gives the new type a short name, so `process_t p;` works like `int x;`. Everything about *one* process lives in *one* element of `procs[]`, instead of being spread over several parallel arrays.

```
typedef struct {
    int id;         /* 1, 2, 3, ...  (P1, P2, P3, ...) */
    int arrival;    /* A */
    int burst;      /* B */
    int remaining;  /* CPU time still needed */
    int start;      /* S, -1 = not started yet */
    int finish;     /* C, -1 = not finished yet */
    int queued;     /* RR only: 1 once in the ready queue */
} process_t;
```

Fields with a dot. `procs[i]` is one whole struct; `.arrival` selects one field. Read `procs[i].arrival` as "the arrival time of process number `i`".

```
procs[i].id      = i + 1;
procs[i].arrival = table[i][0];
```

In sched_sim.c: `load_workload()`. A second struct, `slice_t`, stores one bar of the Gantt chart.

4 Pointers to structs

`p = &procs[i]`, then `p->field`. `&procs[i]` is the address of element `i`; after this line `p` *points to* that process. Use `.` on a struct and `->` on a *pointer* to a struct: `p->arrival` is short for `(*p).arrival`.

```
process_t *p = &procs[i];
if (p->arrival > time || p->remaining == 0) {
    continue;                  /* not ready */
}
```

A function that returns a pointer, or NULL. `pick_fcfs()` returns the address of the chosen process. `NULL` means "points to nothing": here, no process is ready. Always check for

`NULL` before using the pointer.

```
static process_t *pick_fcfs(int time) {
    process_t *best = NULL;      /* no candidate yet */
    ...
    return best;                 /* a process, or NULL */
}
```

A pointer parameter changes the real process. `run_slice()` receives a pointer, so its writes reach the element in `procs[]`. If it received a `process_t` (a copy), every change would be lost when it returned.

```
static void run_slice(process_t *p, int from, int to) {
    ...
    p->remaining -= to - from;
    if (p->remaining == 0) {
        p->finish = to;
    }
```

5 Loops, conditions and operators

Code	Meaning
`continue;`	Jump to the next round of the loop. Used to skip processes that are not ready.
`while (finished < n) { ... }`	Repeat while the condition is true. Something inside must change it, or the loop never ends.
`a || b  a && b  !a`	"or", "and", "not". In C, `0` is false and every other number is true.
`best == NULL || ...`	*Short-circuit*: C stops as soon as the answer is known, so when `best` is `NULL` it never reads `best->arrival`, which would crash.
`c ? a : b`	A short if/else that gives a value: `a` if `c` is true, else `b`.
`x++  x += y  x -= y`	`x = x + 1`, `x = x + y`, `x = x - y`.
`(q_head + q_count) % MAX_PROCS`	`%` is the remainder. It keeps a queue index between 0 and 9: after slot 9 it wraps to slot 0, so the queue reuses the array in a circle.

```
int quantum = (argc == 4) ? atoi(argv[3]) : 2;   /* typed quantum, or 2 */
```

6 Command line, strings and printf

argc and argv. For `./sched_sim rr lab 4`, `argc` is 4 and `argv` holds `"./sched_sim"`, `"rr"`, `"lab"` and `"4"`. Every argument is text, even `4`; `atoi()` converts text to an `int` (and gives `0` for text that is not a number).

Compare strings with strcmp, never with ==. `strcmp(a, b)` returns `0` when the strings are equal; `==` would compare where the strings are stored, not their letters.

```
if (strcmp(policy, "rr") == 0) { ... }
```

printf widths line up the table. A number between `%` and the letter is a minimum width: `%7d` is right-aligned in 7 characters, `%-7d` left-aligned, and `%.2f` prints 2 decimals.

Integer division. Dividing two `int`s drops the fraction: `35 / 4` is `8`. Converting first keeps it: `(double)35 / 4` is `8.75`. That is why the averages use `(double)`.

7 Quick check

1. In `lab_workload`, what are `lab_workload[2][0]` and `lab_workload[2][1]`, and which process is that?
2. `p` points to `procs[1]`. Write two ways to read its burst.
3. Why must `best == NULL` come *before* `p->arrival < best->arrival` in the condition?

Answers: (1) `3` and `6`: P3 arrives at 3 ms and needs 6 ms. (2) `p->burst` or `procs[1].burst` (also `(*p).burst`). (3) Short-circuit: when `best` is `NULL` the right-hand side is never evaluated, so `best->arrival` is never read through a `NULL` pointer.